🛡️ Module 11: Reverse Engineering, Fuzzing and Binary Exploitation

```
# Exercise 14: Fuzzing an Application with American Fuzzy Lop (AFL)
```

Scenario

American Fuzzy Lop (AFL) is an advanced, coverage-guided genetic fuzzer that uses compile-time instrumentation (`afl-gcc`) and genetic algorithms to discover complex code paths, logic flaws, and memory corruption bugs in software binaries.

Key technical workflows in AFL fuzzing:
1. **Target Instrumentation & Binary Protection Disabling**: Compiling target C programs (**aflfirsttest.c**) with memory protections disabled (`-fno-stack-protector -z execstack`) via standard GCC (**aflfirsttest-gcc**) and AFL instrumented wrapper (**aflfirsttest-afl**) via **afl-gcc**.
2. **Binary Security Auditing**: Verifying security mitigations (RELRO, Stack Canaries, NX bit, PIE) using Pwntools **checksec**.
3. **Test Case Corpus Design**: Crafting minimal seed test cases in `./testcases` that maximize execution branch coverage across program validation paths.
4. **Kernel Dump Setup & Fuzzing Engine**: Configuring `/proc/sys/kernel/core_pattern` (`echo core > /proc/sys/kernel/core_pattern`) to allow proper crash detection, running **afl-fuzz -i ./testcases -o ./results ./aflfirsttest-afl**, monitoring unique crashes, total paths, CPU loads, and execution speeds (exec/sec).
5. **Crash Reproduction & Analysis**: Extracting unique crashing inputs from `./results/crashes/` and piping them into target binaries (`cat id:000000... | ./aflfirsttest-gcc`) to reproduce Segmentation Faults (Signal 11).

In this exercise, we log into **Ubuntu 20.04 Fuzzing** VM (**student** / **studentpassword**), disable ASLR (`kernel.randomize_va_space=0`), write vulnerable authentication program **aflfirsttest.c**, audit security protections with `checksec`, compile instrumented binary **aflfirsttest-afl**, construct initial test case corpus (`testcases`), configure kernel core pattern, run `afl-fuzz`, monitor crash metrics, and reproduce crashes in `./results/crashes/`.

Objective

The objective of this lab is to help students learn how to:
- Authenticate to Ubuntu 20.04 target VM (**student** / **studentpassword**)
- Disable Address Space Layout Randomization: **sudo sysctl kernel.randomize_va_space=0**
- Write vulnerable C authentication source file **aflfirsttest.c** using `gets()`
- Compile binary without canary or NX protections: **gcc -fno-stack-protector -z execstack aflfirsttest.c -o aflfirsttest-gcc**
- Verify binary security settings using Pwntools **checksec aflfirsttest-gcc**
- Compile AFL-instrumented target binary: **afl-gcc -fno-stack-protector -z execstack aflfirsttest.c -o aflfirsttest-afl**
- Prepare test case directory (`./testcases`) with 3 distinct credential input files
- Configure kernel core dump pattern: **echo core > /proc/sys/kernel/core_pattern**
- Launch AFL fuzzing engine: **afl-fuzz -i ./testcases -o ./results ./aflfirsttest-afl**
- Inspect AFL UI dashboard metrics (Total paths, Unique crashes, Exec speed, CPU load)
- Identify AFL version (**1.96b**)
- Replicate unique crash inputs against compiled target binary in `./results/crashes`

Task 1: Write Vulnerable Application, Compile with AFL, Execute AFL Fuzzer, and Analyze Crash Input

In this task, we write `aflfirsttest.c`, compile with GCC and `afl-gcc`, audit binary security with `checksec`, construct input test cases, run `afl-fuzz`, monitor unique crashes, and reproduce segmentation faults.

Lab Tasks

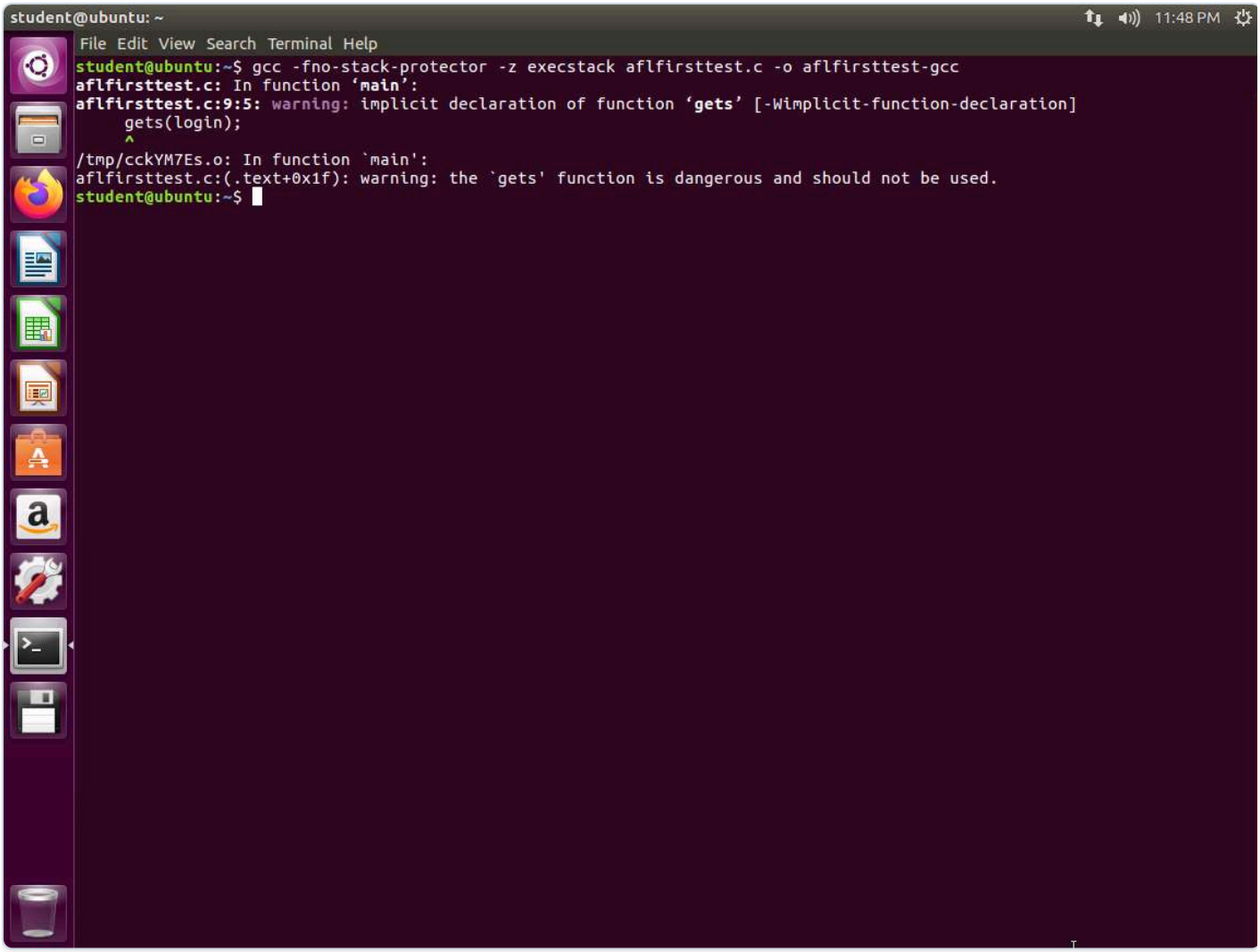
1. Log into **Ub20 Fuzzing** VM using credentials (**student** / **studentpassword**). Open terminal and disable ASLR:
   sudo sysctl kernel.randomize_va_space=0

2. Create vulnerable C program **aflfirsttest.c**:

```
#include <stdio.h>
#include <string.h>

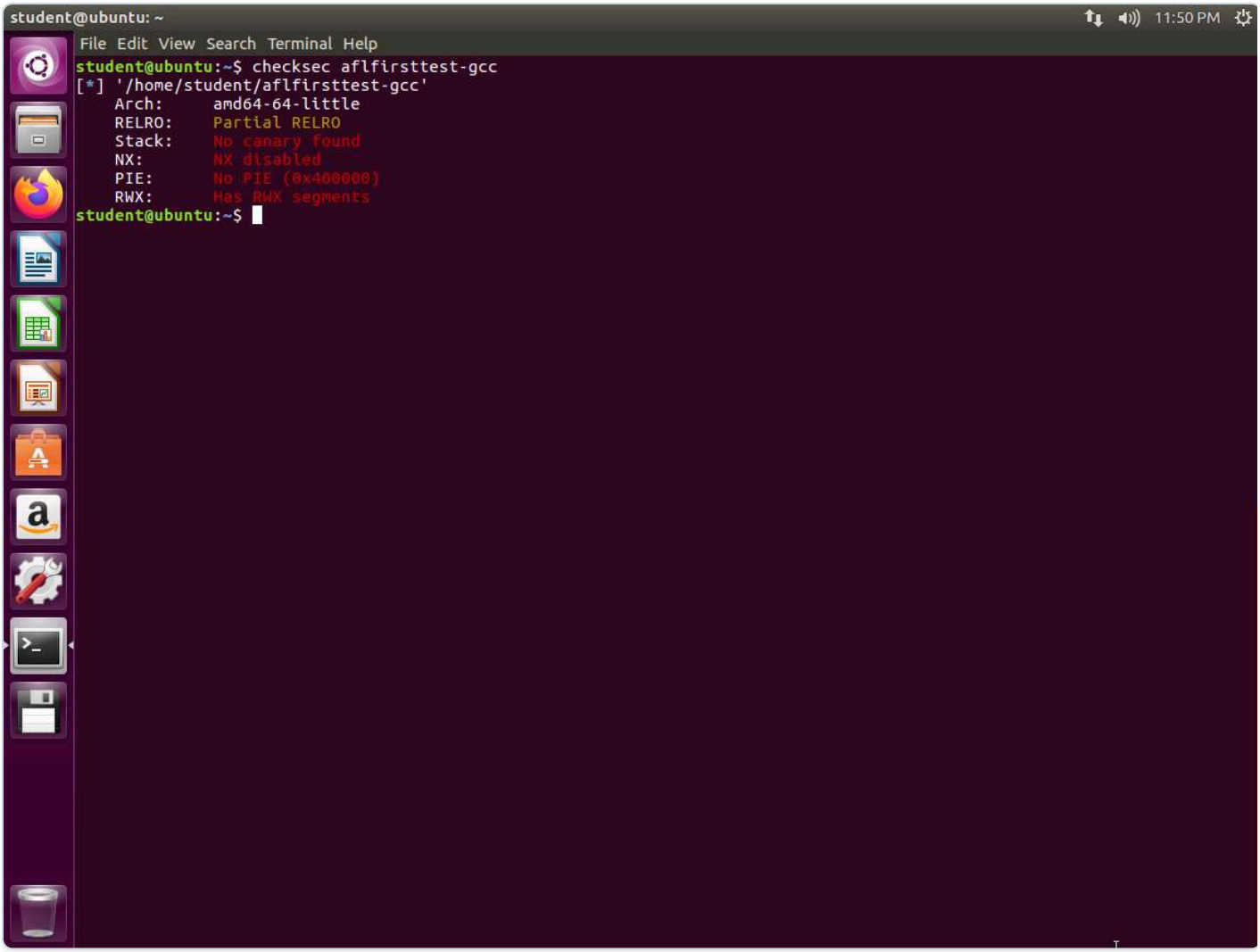
int main(void)
{
    char login[32];
    char passwd[32];
    printf("Login: \n");
    gets(login);
    printf("Password: \n");
    gets(passwd);
    if (strcmp(login, "root") == 0) {
        if (strcmp(passwd, "1qazxsw2") == 0) {
            printf("Access Granted.\n");
            return 0;
        }
    }
    printf("Access Denied.\n");
    return 1;
}
```

1. Compile with standard GCC disabling stack protections and setting executable stack:
   gcc -fno-stack-protector -z execstack aflfirsttest.c -o aflfirsttest-gcc

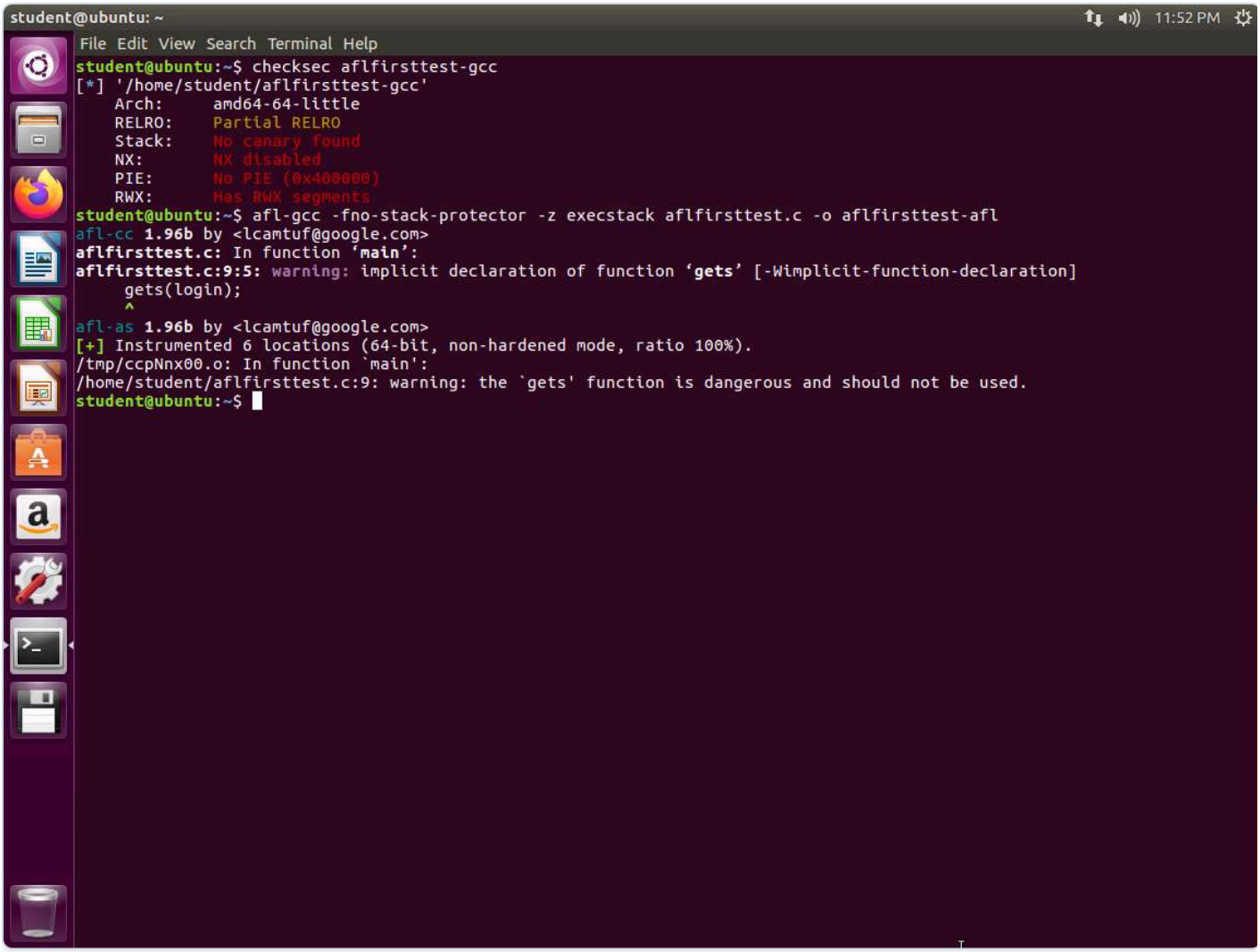


1. Verify disabled security protections using Pwntools checksec:

   checksec aflfirsttest-gcc



1. Compile instrumented binary with `afl-gcc`:
   afl-gcc -fno-stack-protector -z execstack aflfirsttest.c -o aflfirsttest-afl



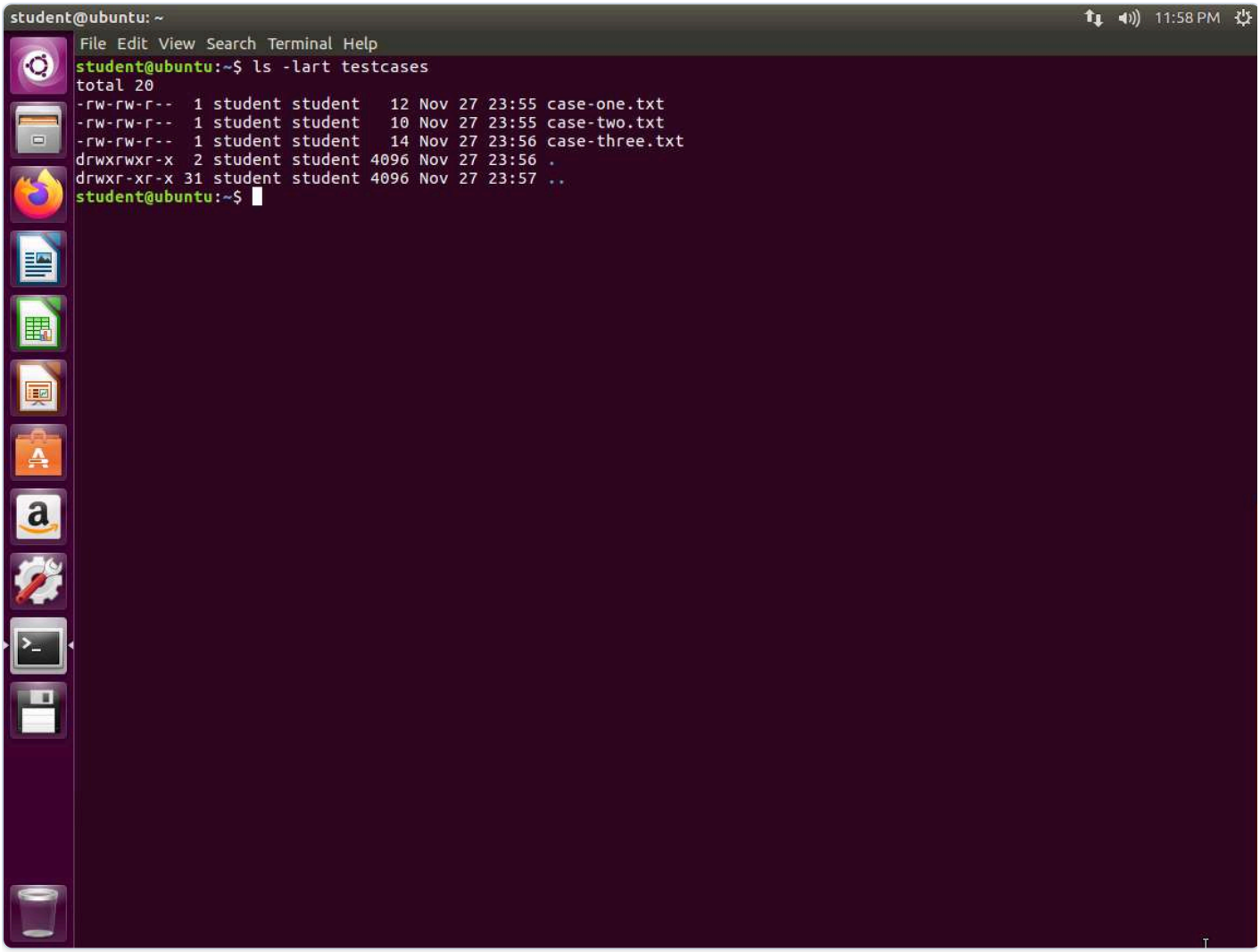
1. Review test case corpus requirements for program branch paths.

File One Name	File Two Name	File Three Name
case-one.txt	case-two.txt	case-three.txt
admin admin	root toor	root 1qazxsw2

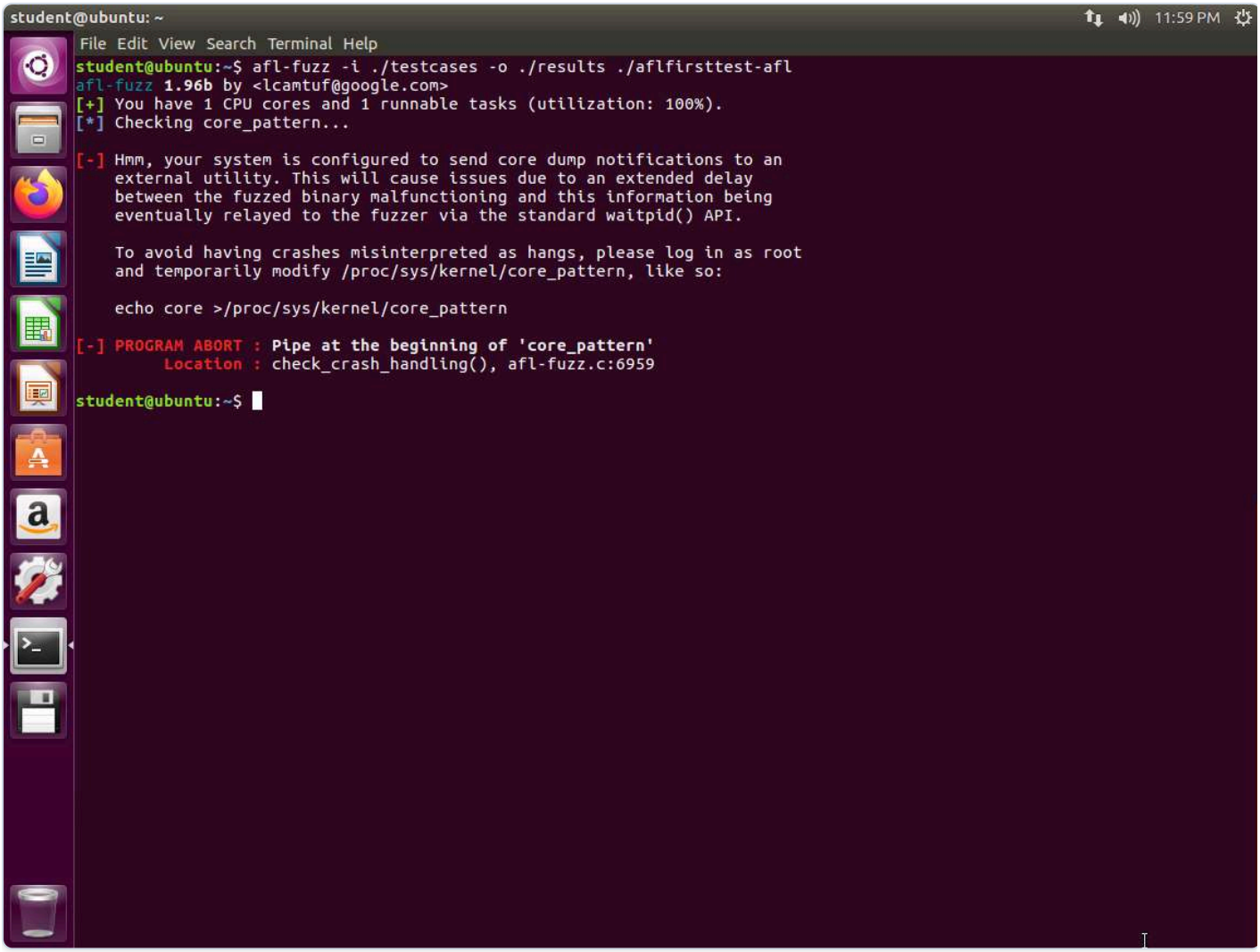
1. Create `testcases` directory and verify input files:

   mkdir testcases results

   ls -lart testcases



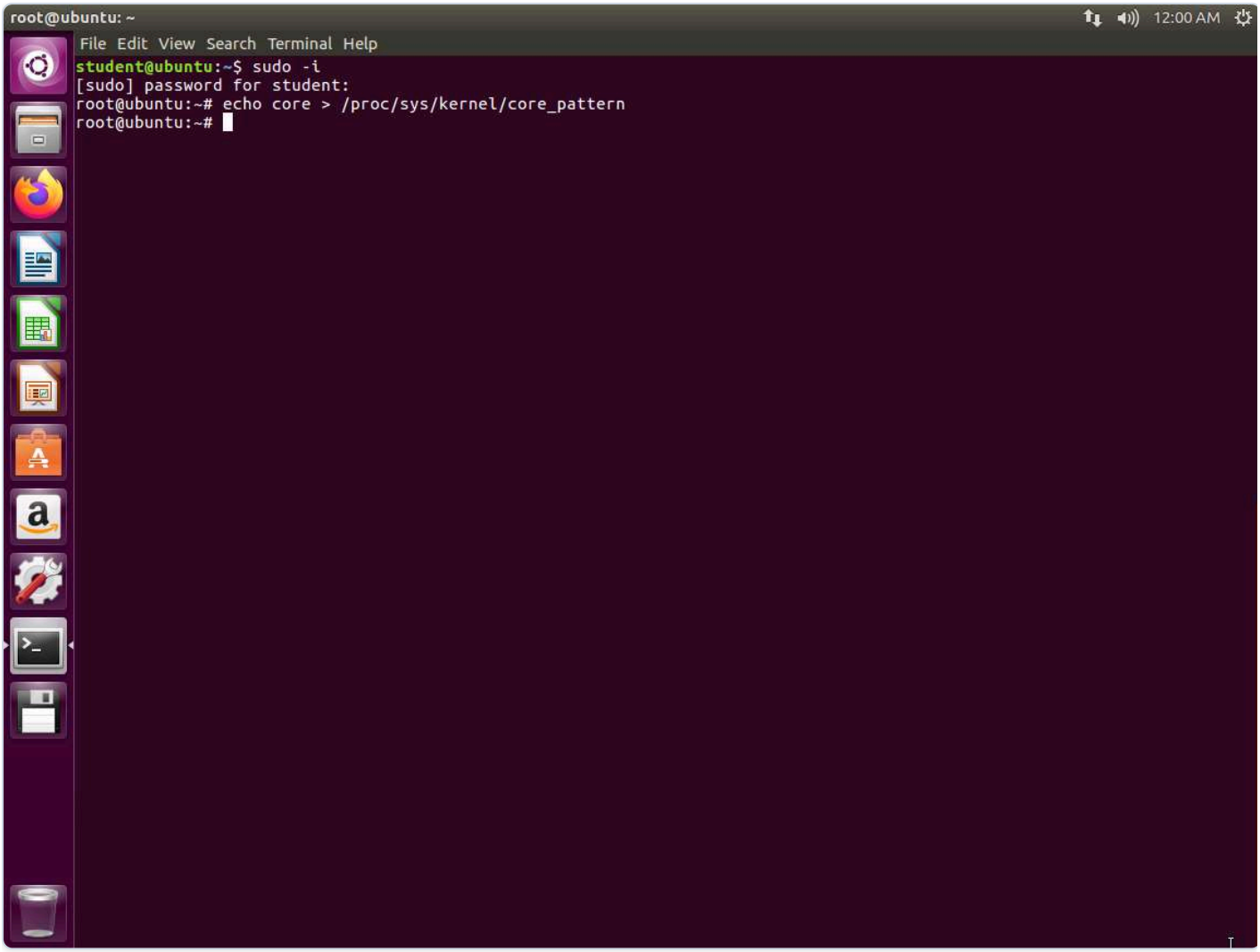
1. Execute `afl-fuzz` engine:
   afl-fuzz -i ./testcases -o ./results ./aflfirsttest-afl



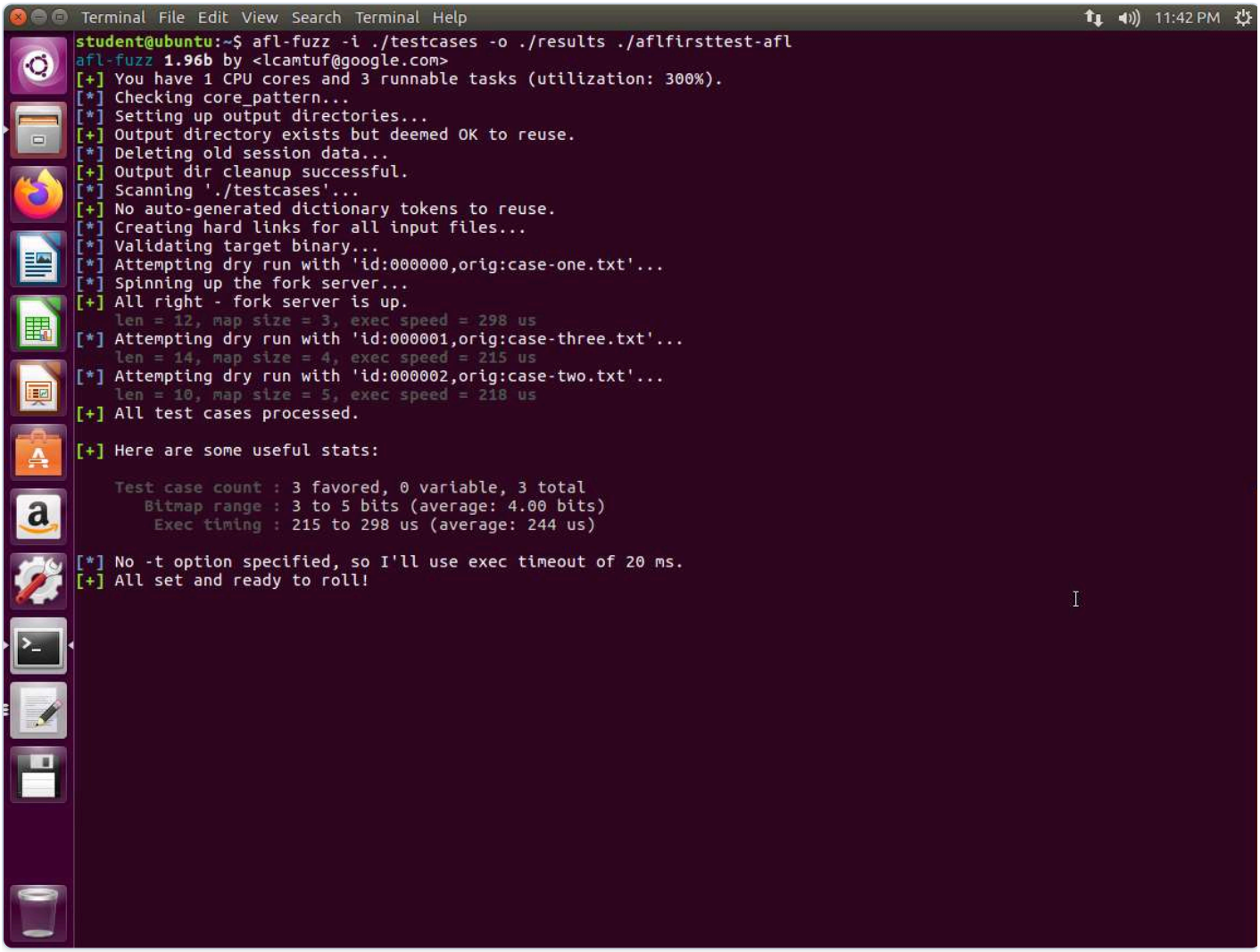
1. Open root shell and set core dump pattern:

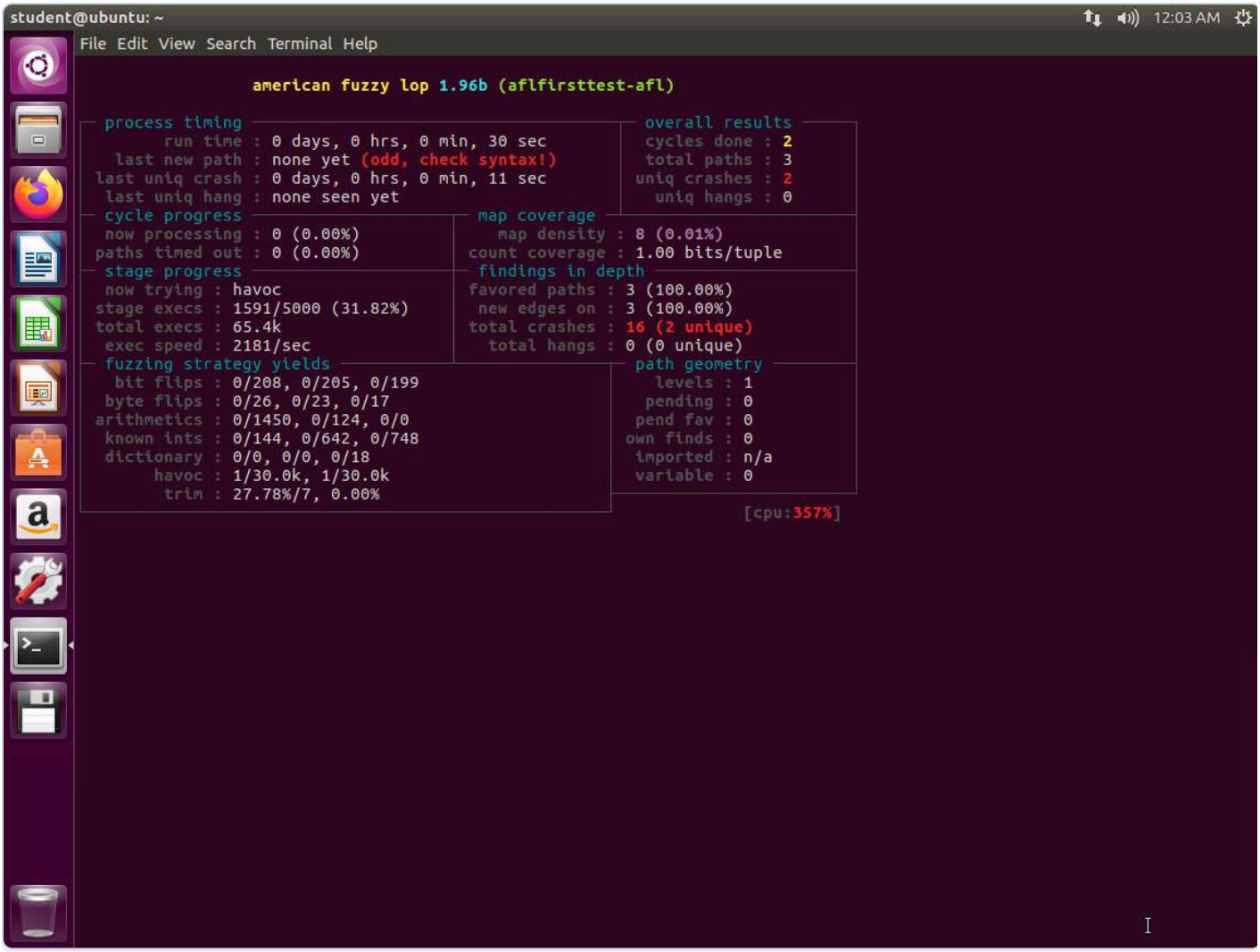
   sudo -i

   echo core > /proc/sys/kernel/core_pattern

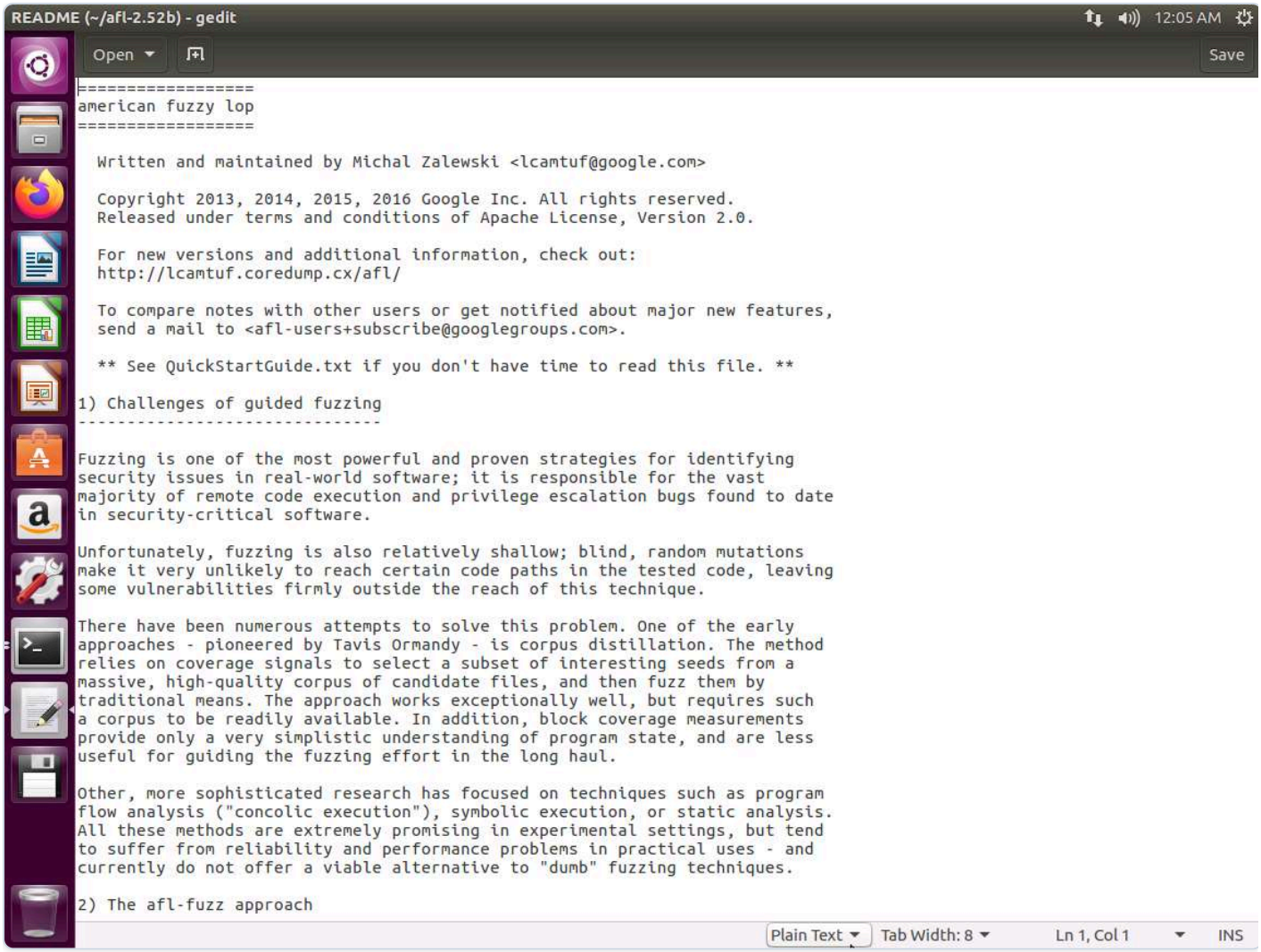


1. Re-run `afl-fuzz` command in original window:
   afl-fuzz -i ./testcases -o ./results ./aflfirsttest-afl

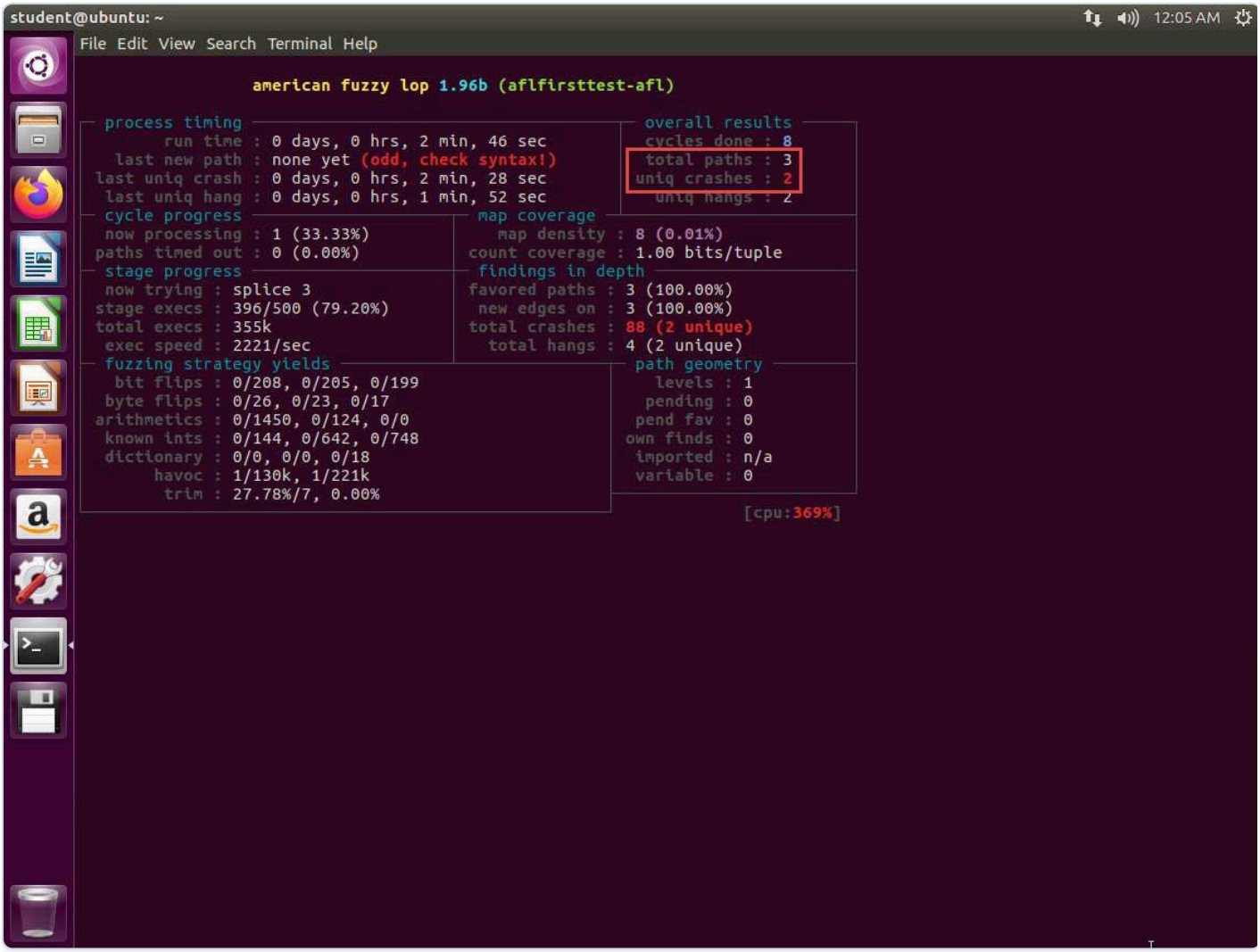

Terminal File Edit View Search Terminal Help
11:42 PM
student@ubuntu:~$ afl-fuzz -i ./testcases -o ./results ./aflfirsttest-afl
afl-fuzz 1.96b by <lcamtuf@google.com>
[+] You have 1 CPU cores and 3 runnable tasks (utilization: 300%).
[*] Checking core_pattern...
[*] Setting up output directories...
[+] Output directory exists but deemed OK to reuse.
[*] Deleting old session data...
[+] Output dir cleanup successful.
[*] Scanning './testcases'...
[+] No auto-generated dictionary tokens to reuse.
[*] Creating hard links for all input files...
[*] Validating target binary...
[*] Attempting dry run with 'id:000000,orig:case-one.txt'...
[*] Spinning up the fork server...
[+] All right - fork server is up.
len = 12, map size = 3, exec speed = 298 us
[*] Attempting dry run with 'id:000001,orig:case-three.txt'...
len = 14, map size = 4, exec speed = 215 us
[*] Attempting dry run with 'id:000002,orig:case-two.txt'...
len = 10, map size = 5, exec speed = 218 us
[+] All test cases processed.
[+] Here are some useful stats:
Test case count : 3 favored, 0 variable, 3 total
Bitmap range : 3 to 5 bits (average: 4.00 bits)
Exec timing : 215 to 298 us (average: 244 us)
[*] No -t option specified, so I'll use exec timeout of 20 ms.
[+] All set and ready to roll!



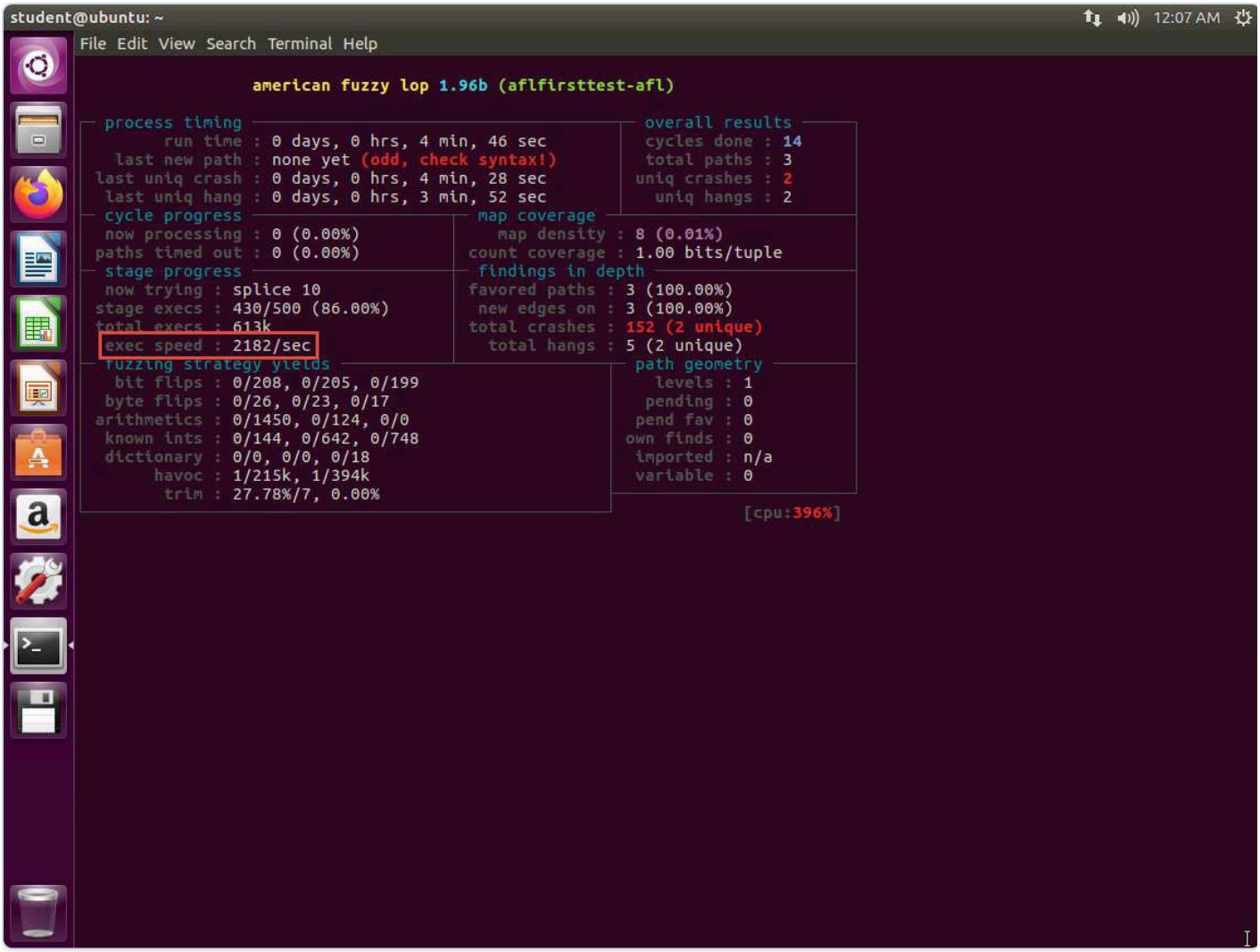
1. Inspect AFL README documentation in `/home/student/afl-2.52b`.



1. Inspect Total Paths and Unique Crashes dashboard panels.

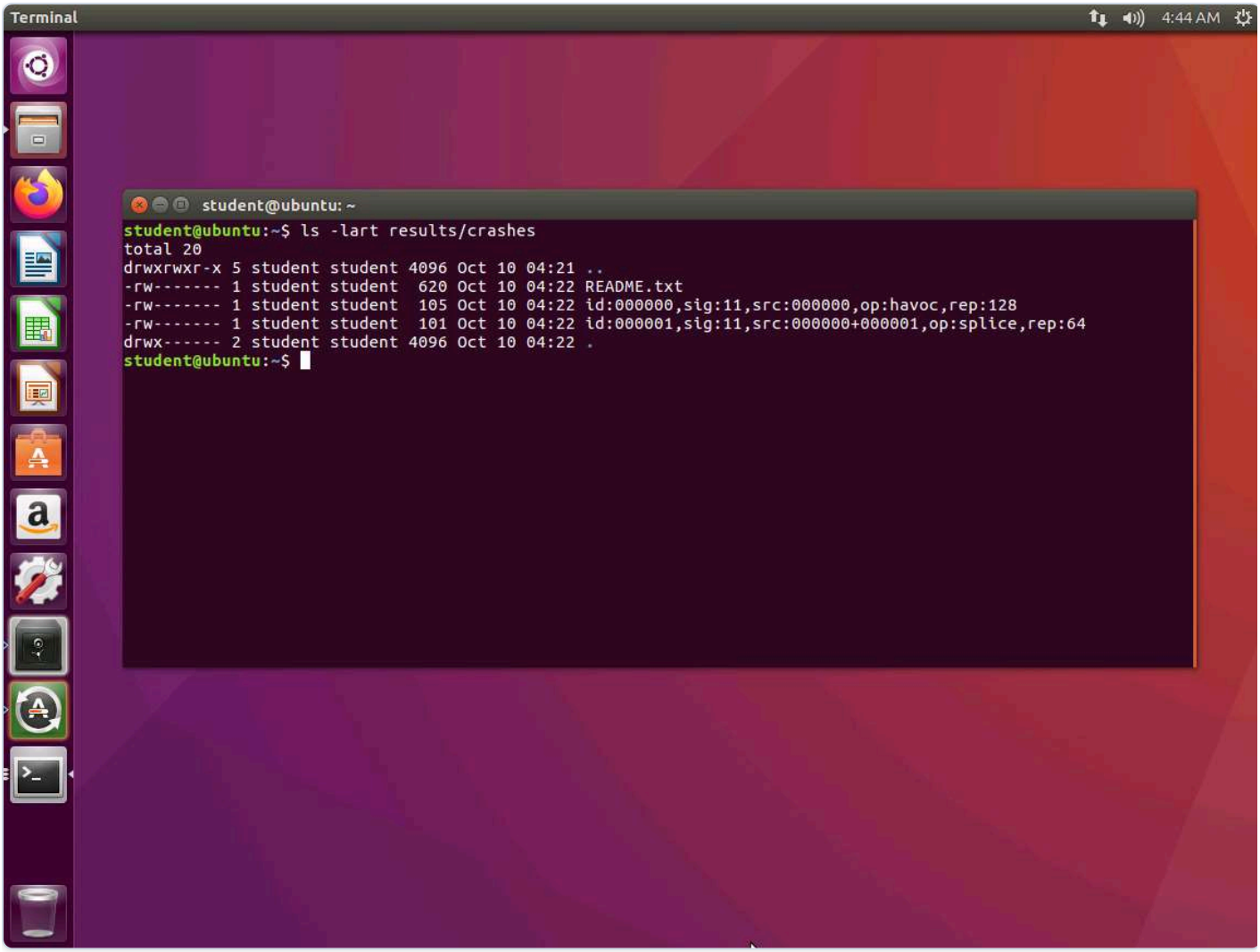


1. Inspect CPU Load and Execution Speed metrics.



1. List unique crash test case files in `./results/crashes`:

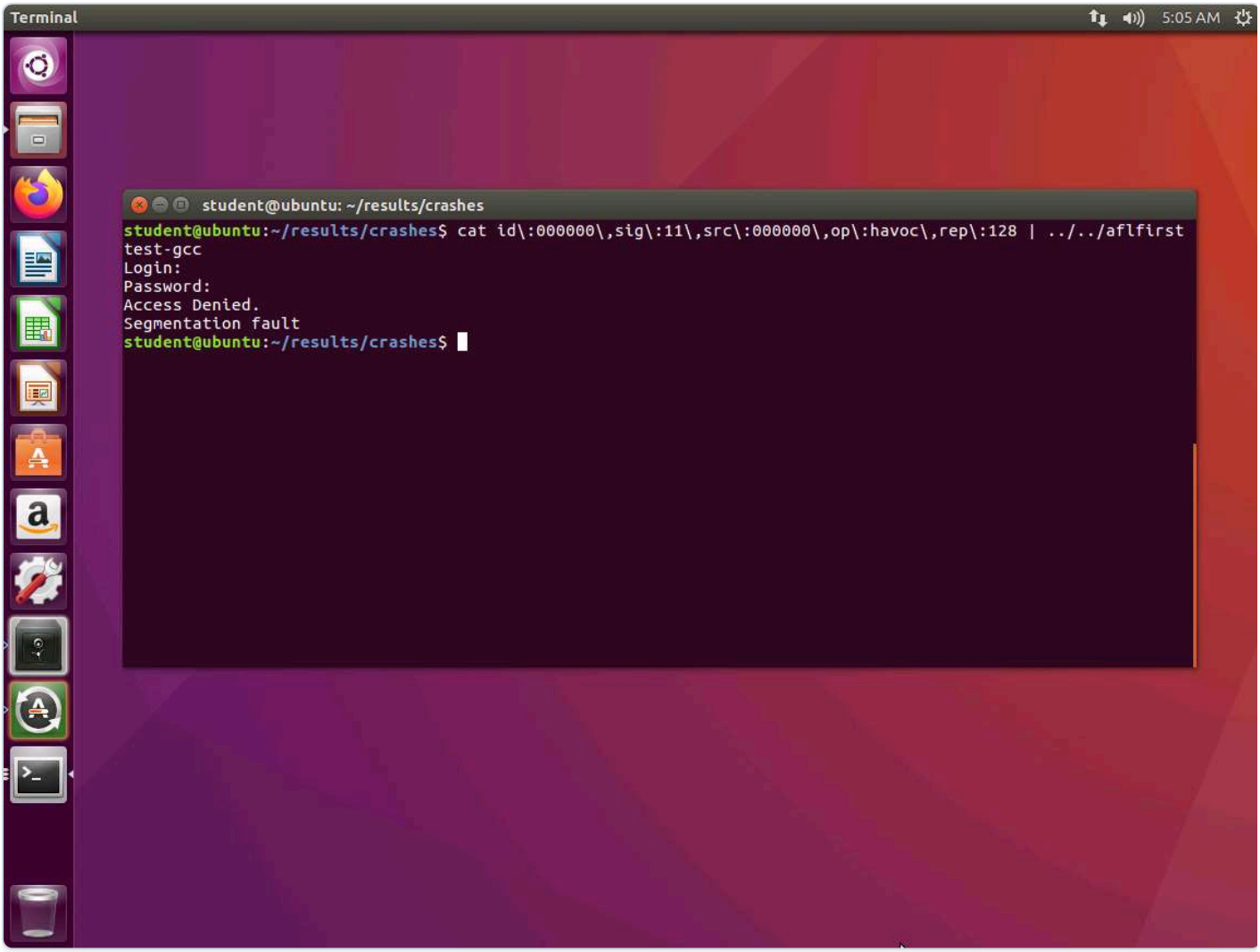
   ls -lart results/crashes



1. Pipe crashing input file into target binary to reproduce Segmentation Fault (Signal 11):

   cd ./results/crashes

   cat id:000000,sig:11,src:000000,op:havoc,rep:128 | ../../aflfirsttest-gcc



This concludes the lab exercise.

> **IMPORTANT:** ❓ **Question 11.14.1.1** Perform fuzzing using the American Fuzzy Lop (AFL) tool in the Ub20 Fuzzing machine. Enter the version number of the American Fuzzy Lop (AFL) tool. [!SUCCESS] ✅ **Answer:** **1.96b**